

New Incremental Sprint 5 Requirements Package

1. New Incremental Sprint Scope Summary

This New Incremental Sprint is limited to two increments:

1. replacing Gemini and all Gemini-specific dependencies with the approved local Qwen model arrangement; and
2. introducing a manually assigned and Backend-authorised Admin role limited to adding new Interest Tags to the existing Interest catalogue.

Qwen3.5-9B is the approved primary model. Qwen3.5-4B is the manually configured development and low-spec alternative model. It is not a second primary model, an automatic failover model, or a runtime-routing target. Ollama is the current local inference runtime, while vLLM, SGLang, and llama.cpp remain future deployment options rather than current implementation commitments.

The existing Backend-only AI architecture remains in force. Flutter does not directly access the model or inference runtime. The Backend performs authentication, authorisation, privacy checks, visibility checks, data minimisation, output parsing, output validation, business-rule enforcement, and deterministic Backend fallback. The deterministic Backend fallback returns a controlled result when model inference is unavailable or invalid and does not automatically activate another model. Qwen does not directly access Firestore or modify application state.

All recommendation candidates, weights, scores, and final ordering remain controlled by the deterministic Backend recommendation algorithm. Qwen may only explain, summarise, or conversationally present Backend-approved recommendation results. It does not add candidates, modify weights, override scores, change final ordering, or independently create, select, or assign Interest Tags.

The Admin increment introduces a manually assigned and Backend-authorised Admin role limited to adding new Interest Tags through a protected Backend-controlled Create operation. An Interest Tag is an item in the project's existing Interest catalogue. The Admin does not perform Interest content review, approval, moderation, or content-rejection decisions; does not formally administer Interest Read operations; does not update, rename, merge, deactivate, reactivate, delete, archive, or restore Interest content; and does not perform User Account Administration. This restriction does not prevent the Backend from operationally rejecting and recording an unauthorised, invalid, or out-of-scope Interest Tag Create request.

Successfully added Interest Tags become available to existing features that already retrieve, display, select, or use the Interest catalogue, including existing Check-in and Interest-based Event and Friend Recommendation functions, without redesigning those functions. Existing

Interest-consuming features may display or use a newly added Interest Tag only through their existing catalogue integration and approved business rules.

Creating an Interest Tag does not automatically assign it to users, Events, Check-ins, recommendations, AI input, or historical records. It does not modify existing user selections, Event associations, Check-in records, Profiles, recommendation weights, scores, final ordering, Going or Interested states, friendships, or historical application data.

This package defines design-level requirements, responsibility boundaries, and preparation specifications. It does not claim completed software implementation. No Admin UI is required in the current Sprint. Any earlier Project Plan reference to an Admin prototype, configuration interface, dashboard, mobile page, or complete Admin UI workflow is Deferred.

Existing Check-in, Privacy, Event, Friends Activity, Special Care, Friend Search, Recommendation, Homepage, Persona, Going, and Interested business rules remain frozen. The only approved catalogue-level synchronisation is that a successfully added Interest Tag becomes available to existing features that already consume the Interest catalogue under their existing approved rules.

2. Consolidated Incremental Change Register

Change ID	Feature Area	Change Type	Consolidated Change
NIS-CH 01	AI Provider and Model Migration	Modified and Removed	Remove Gemini, Gemini API, Gemini Provider Adapter, Gemini credentials, quota, billing, free-tier, rate-limit, SDK, and cloud-provider dependencies. Adopt Qwen3.5-9B as the primary model and Qwen3.5-4B as the manually configured development and low-spec alternative model.
NIS-CH 02	Local AI Runtime and Backend Integration	Modified	Use Ollama as the current local inference runtime and Qwen Local Model Adapter as the single Backend integration abstraction. Preserve Backend-only access, data minimisation, validation, deterministic Backend fallback, and no direct model-to-database access.

NIS-CH 03	Backend Admin Identity and Access	Added	Add a manually assigned Admin role using existing Firebase Authentication and Firebase custom claims. Allow a small set of manually approved Admins. Admin authority is limited to adding new Interest Tags to the existing Interest catalogue, and every permitted operation requires Backend authentication and authorisation.
NIS-CH 04	Interest Tag Addition	Added	Allow an authorised Admin to add a new Interest Tag to the existing Interest catalogue through a protected Backend-controlled Create operation. After successful creation, the tag becomes available to existing features that already consume the Interest catalogue under their existing approved rules. No separate Event-specific or Activity-specific tag entity is introduced. The operation requires authentication, authorisation, entity-specific validation, and creation audit logging. No Interest content review, approval, content-rejection decision, moderation, formal Read administration, update, rename, merge, deactivation, reactivation, deletion, archiving, or restoration is included. Operational rejection of an unauthorised, invalid, or out-of-scope Create request remains required.

3. User Stories and Acceptance Criteria

NIS-US01 — Continue Approved AI Functions with Local Qwen Models

Change Type: Modified and Removed

User Story

As a user, I want approved AI-supported functions to continue through a locally deployed model so that they do not depend on Gemini credentials, quotas, billing, or cloud-provider availability.

Acceptance Criteria

- **NIS-AC01:** Qwen3.5-9B shall be the project's primary model.
- **NIS-AC02:** Qwen3.5-4B shall be used only as the manually configured development and low-spec alternative model and shall not be presented as a second primary model, an automatic failover model, or a runtime-routing target.
- **NIS-AC03:** Ollama shall be the current local inference runtime, and the Backend shall communicate with it through Qwen Local Model Adapter.
- **NIS-AC04:** Gemini API, credentials, quotas, billing, free-tier assumptions, provider-specific SDK bindings, and Gemini Provider Adapter shall no longer be required.
- **NIS-AC05:** Existing approved AI explanations, summaries, constrained suggestions, recommendation support, and Persona-based responses shall remain within their previously approved business scope. All recommendation candidates, weights, scores, and final ordering shall be generated by the deterministic Backend recommendation algorithm. Qwen may only explain, summarise, or conversationally present Backend-approved recommendation results and shall not add candidates, independently create or select Interest Tags, modify weights, override scores, or change final ordering.
- **NIS-AC06:** Flutter shall not directly call Qwen3.5-9B, Qwen3.5-4B, Ollama, vLLM, SGLang, llama.cpp, or a model endpoint.
- **NIS-AC07:** No automatic model switching, automatic model failover, runtime routing, load-based model selection, per-feature model assignment, or user-selectable model option shall be introduced.

NIS-US02 — Receive Validated AI Results and Safe Fallback

Change Type: Modified

User Story

As a user, I want AI results to remain controlled by the Backend and supported by a safe fallback so that invalid or unavailable model output does not compromise application behaviour.

Acceptance Criteria

- **NIS-AC08:** All AI requests shall pass through the Backend, which shall perform authentication, authorisation, privacy checks, visibility checks, data minimisation, purpose limitation, and request validation.
- **NIS-AC09:** The Backend shall provide the model only with the minimum permitted data required for the current approved request. Any inconsistent earlier sponsor or Project Plan wording suggesting comprehensive, broad, or unrestricted data collection is superseded by this requirement.
- **NIS-AC10:** The Backend shall validate expected structure, required values, allowed identifiers, approved candidates, permitted information, and business-rule compliance before accepting model output.
- **NIS-AC11:** Unknown identifiers, unsupported candidates, malformed output, restricted information, privacy violations, visibility violations, and unsupported state changes shall be rejected.

- **NIS-AC12:** A deterministic Backend fallback shall be used when the model or runtime is unavailable, times out, fails, or returns invalid output. It shall return a controlled deterministic result without automatically activating another model. Core non-AI functions shall remain usable.

NIS-US03 — Add an Interest Tag as an Authorised Admin

Change Type: Added

User Story

As an authorised Admin, I want to add a new Interest Tag to the existing Interest catalogue through a protected Backend-controlled operation so that the available Interest options can be extended consistently across existing Interest-consuming features.

Acceptance Criteria

- **NIS-AC13:** Admin shall use the existing Firebase Authentication, and Admin authority shall be verified by the Backend using Firebase custom claims.
- **NIS-AC14:** Admin assignment shall be performed manually by the Backend or project owner, and the system may support a small set of manually approved Admins.
- **NIS-AC15:** An authorised Admin shall be able to add a new Interest Tag to the existing Interest catalogue through the protected Backend-controlled Create operation.
- **NIS-AC16:** A new Interest Tag shall pass entity-specific Backend validation before creation is accepted.
- **NIS-AC17:** After successful validation, the Backend shall create the new Interest Tag within the existing Interest catalogue. The new tag shall become available to current features that already retrieve, display, select, or use that catalogue under their existing approved rules. Creation shall not automatically assign the tag to users, Events, Check-ins, recommendations, AI input, or historical records.
- **NIS-AC18:** For a successful Interest Tag creation, the audit record shall include Admin identity, Create operation, target entity, created target record, timestamp, the created tag value where permitted, and successful outcome. For a failed or operationally rejected Create request, the audit record shall include Admin identity where available, attempted Create operation, target entity, attempted tag value where permitted, timestamp, and failed or operationally rejected outcome, without claiming that a target record was created. The audit shall not record Interest content-review, approval, moderation, or content-rejection decisions.

4. Added and Modified Use Cases

4.1 Added or Consolidated Use Cases

NIS-UC01 — Process a Backend-Controlled Qwen Request

Primary Actor: User

Supporting Components: Backend, Qwen Local Model Adapter, Ollama, Qwen3.5-9B, or Qwen3.5-4B as the manually configured development and low-spec alternative model

Preconditions

- The user is authenticated where required by the existing feature.
- The request belongs to an approved AI-supported purpose.
- The Backend can determine the permitted data and candidates for the request.

Trigger

- The user initiates an existing approved AI-supported function.

Main Flow

1. Flutter sends the authenticated request to the Backend.
2. The Backend identifies the approved request purpose.
3. The Backend verifies authorisation, privacy, visibility, purpose limitation, and input validity.
4. The Backend retrieves and minimises only the permitted data required for the request.
5. The Backend prepares any recommendation candidates, weights, scores, and final ordering through the deterministic Backend recommendation algorithm where applicable.
6. The Backend sends only approved structured context or approved recommendation results through Qwen Local Model Adapter to the current Ollama runtime.
7. The approved Qwen model produces a constrained explanation, summary, conversational presentation, or other previously approved result.
8. The Backend parses and validates the result.
9. The Backend returns only the validated result to Flutter.

Alternative Flow — Invalid or Unavailable Model Result

1. The model or runtime is unavailable, times out, fails, or returns invalid output.
2. The Backend rejects the invalid result.
3. The Backend uses the deterministic fallback defined for the approved function.
4. The application returns a controlled result without automatically switching to another model or creating unsupported data or state.

Postconditions

- Flutter receives only a Backend-approved result.
- Qwen has not added recommendation candidates, independently created or selected Interest Tags, modified weights, overridden scores, or changed final ordering.
- The model does not directly access application data or modify application state.

Scope Boundary

- Qwen3.5-4B is the manually configured development and low-spec alternative model, not an automatic failover from Qwen3.5-9B.
- No automatic routing between Qwen3.5-9B and Qwen3.5-4B is introduced.
- No direct Flutter-to-model or model-to-database connection is introduced.
- Existing AI business outcomes remain unchanged.

NIS-UC02 — Add an Interest Tag

Primary Actor: Admin

Preconditions

- The Admin is authenticated through Firebase Authentication.
- The Backend verifies the Admin custom claim.
- The Admin uses a restricted internal script or command-line tool that calls the protected internal Admin API.
- The requested operation concerns creation of a new Interest Tag within the existing Interest catalogue only.

Trigger

- The Admin submits a new Interest Tag for creation.

Main Flow

1. The Admin submits the new Interest Tag.
2. The Backend verifies authentication and Admin authority.
3. The Backend confirms that the requested operation is Add or Create only.
4. The Backend performs entity-specific validation.
5. If validation passes, the Backend creates the new tag within the existing Interest catalogue.
6. The Backend makes the updated catalogue available through the existing catalogue retrieval paths.
7. The Backend records the creation audit result.
8. The Backend returns a controlled success result.

Alternative Flow — Invalid, Unauthorised, or Out-of-Scope Create Request

1. Authentication, authorisation, operation-scope validation, or entity-specific validation fails.
2. The Backend operationally rejects the Create request.
3. No Interest Tag is created or changed.
4. The Backend records the failed or rejected operational outcome. This record does not constitute an Interest content-review approval or rejection decision and does not claim that a target record was created.

Postconditions

- A valid new Interest Tag is created within the existing Interest catalogue, or the invalid, unauthorised, or out-of-scope request is operationally rejected.
- The successfully created Interest Tag becomes available to existing features that already consume the Interest catalogue under their existing approved rules.
- No existing Interest is updated, renamed, merged, deactivated, reactivated, deleted, archived, restored, subjected to content review, approved, subjected to a content-rejection decision, or moderated.
- No user, Event, Check-in, recommendation, AI input, Profile, or historical record is automatically assigned or modified.

Scope Boundary

- Admin authority is limited to adding new Interest Tags to the existing Interest catalogue.
- “Interest Tag” refers to an item in the existing general Interest catalogue and does not define a separate Event-specific or Activity-specific entity.
- The Use Case does not include Interest Read as a formal administration function.
- The Use Case does not include Update, Rename, Merge, Deactivate, Reactivate, Delete, Archive, Restore, content review, approval, content-rejection decisions, or moderation.
- Existing Interest-consuming features may display or use the new tag through their existing catalogue integration and approved business rules.
- No tag is automatically assigned to users, Events, Check-ins, recommendations, AI input, or historical records.
- Existing Check-in, Profile, Event Recommendation, Friend Recommendation, AI, privacy, and visibility logic is not redesigned.
- Excluding content-rejection decisions from the Admin scope does not prevent the Backend from operationally rejecting an unauthorised, invalid, or out-of-scope Create request.

4.2 Modified Existing Use Cases

Existing Use Case	Change Type	New Incremental Sprint Modification
N/A	Modified	Existing approved AI-supported functions shall use Qwen Local Model Adapter, Ollama, and the approved Qwen models instead of Gemini. Their existing business outcomes remain unchanged.
N/A	Modified	Existing Persona-based AI responses shall use the validated Backend-controlled Qwen flow without changing Persona selection, management, or maximum-five rules.

N/A	Modified and Removed	The existing External AI Integration Layer shall replace Gemini Provider Adapter and Gemini request handling with Qwen Local Model Adapter and local-runtime handling while preserving validation and deterministic Backend fallback.
-----	----------------------	---

5. Functional and Non-Functional Requirements

5.1 Functional Requirements

Requirement ID	Change Type	Requirement
NIS-FR01	Modified	Qwen3.5-9B shall be the primary model for approved project AI functions.
NIS-FR02	Added	Qwen3.5-4B shall be available only as the manually configured development and low-spec alternative model and shall not act as an automatic failover from Qwen3.5-9B.
NIS-FR03	Modified	Ollama shall be the current local inference runtime, accessed by the Backend through Qwen Local Model Adapter.
NIS-FR04	Removed	The application shall not require Gemini, Gemini API, Gemini Provider Adapter, Gemini SDK, Gemini credentials, Gemini quota, Gemini billing, or Gemini free-tier availability.
NIS-FR05	Modified	All AI requests and responses shall pass through the Backend; Flutter shall not directly access any model or inference runtime.

NIS-FR06	Modified	The Backend shall apply authentication, authorisation, privacy checks, visibility checks, data minimisation, purpose limitation, request construction, output parsing, and output validation.
NIS-FR07	Modified	The model shall receive only the minimum permitted data and approved results required for the current request. Any inconsistent earlier sponsor or Project Plan wording suggesting comprehensive, broad, or unrestricted data collection shall not apply.
NIS-FR08	Modified	The Backend shall reject invalid structure, unknown identifiers, unauthorised information, unsupported candidates, and business-rule violations.
NIS-FR09	Modified	A deterministic Backend fallback shall remain available when model inference fails or produces an invalid result and shall not automatically activate another model.
NIS-FR10	Modified	Qwen and all inference runtimes shall not directly read, create, update, delete, or query Firestore or another application database.
NIS-FR11	Added	Admin shall use existing Firebase Authentication, manual assignment, and Backend-verified Firebase custom claims for the protected Interest Tag Create operation.
NIS-FR12	Added	The system shall allow a small manually approved set of Admins and shall prevent Client-side declaration of Admin authority.
NIS-FR13	Added	An authorised Admin shall add new Interest Tags to the existing Interest catalogue only through a protected Backend-controlled Create operation.

NIS-FR14	Added	A successfully validated Interest Tag shall be created within the existing Interest catalogue and made available to current features that already consume that catalogue under existing approved rules. Creation shall not automatically modify existing users, Events, Check-ins, recommendations, AI inputs, Profiles, or historical application data.
NIS-FR15	Added	Interest Tag creation shall pass entity-specific Backend validation before acceptance.
NIS-FR16	Added	For a successful Interest Tag creation, the audit record shall include Admin identity, Create operation, target entity, created target record, timestamp, the created tag value where permitted, and successful outcome. For an unauthorised, invalid, or out-of-scope Create request, the audit record shall include Admin identity where available, attempted Create operation, target entity, attempted tag value where permitted, timestamp, and failed or operationally rejected outcome. A failed or rejected request shall not be recorded as having created a target record. The audit shall not introduce Interest content-review, approval, content-rejection, or moderation decisions and shall not include unrelated private-user information.

5.2 Non-Functional Requirements

Requirement ID	Change Type	Requirement
NIS-NFR01	Modified	AI provider abstraction should allow the Backend integration to change inference runtimes without changing Flutter business flows.

NIS-NFR02	Modified	AI-supported functions should remain available through deterministic Backend fallback when local inference is unavailable or invalid, without automatic model failover.
NIS-NFR03	Modified	Model inputs and operational records should minimise sensitive or unrelated user information in accordance with privacy, visibility, purpose-limitation, and Backend-control requirements.
NIS-NFR04	Added	The Interest Tag Create operation shall be protected against unauthorised use through Firebase Authentication, Backend authorisation, and custom-claim verification.
NIS-NFR05	Added	Interest Tag creation should provide sufficient auditability to identify the Admin where available, the successful or attempted Create operation, target entity, relevant created or attempted tag information where permitted, time, and outcome. A successful creation may reference the created target record, while a failed or operationally rejected request shall not claim that a target record was created. Interest content-review, approval, moderation, content-rejection decisions, and unrelated private-user content shall remain outside the audit scope.

6. Business Rules and Security Boundaries

6.1 Business Rules

Rule ID	Change Type	Business Rule
NIS-B R01	Modified	Qwen3.5-9B is the only primary model. Qwen3.5-4B is limited to the manually configured development and low-spec alternative model and is not an automatic failover model.

NIS-B R02	Modified	Ollama is the current runtime, while Qwen Local Model Adapter is the Backend abstraction. They are not AI models or databases.
NIS-B R03	Modified	vLLM and SGLang may be considered later when a suitable NVIDIA GPU is available; Ollama or llama.cpp may continue where no suitable GPU is available.
NIS-B R04	Modified	Every AI request must pass through the Backend, and Flutter cannot directly access the model or runtime.
NIS-B R05	Modified	AI cannot directly access the application database, make permission decisions, or execute critical application-state changes. All recommendation candidates, weights, scores, and final ordering shall be generated by the deterministic Backend recommendation algorithm. Qwen may only explain, summarise, or conversationally present Backend-approved recommendation results and shall not add candidates, independently create, select, or assign Interest Tags, modify weights, override scores, or change final ordering.
NIS-B R06	Added	Admins are manually assigned, authenticated through Firebase Authentication, authorised through Backend-verified Firebase custom claims, and limited to a small approved set.
NIS-B R07	Added	The confirmed Admin scope in this New Incremental Sprint is limited to adding new Interest Tags to the existing Interest catalogue. No separate Event-specific or Activity-specific tag entity and no other Admin-managed authority are introduced.
NIS-B R08	Added	Admin may create a new Interest Tag within the existing Interest catalogue but may not perform a standalone Interest Read administration operation, Update, Rename, Merge, Deactivate, Reactivate, Delete, Archive, Restore, content review, approval, content-rejection decision, or moderation under the authority defined in this Sprint. This restriction does not prevent the Backend from operationally rejecting an unauthorised, invalid, or out-of-scope Create request.

6.2 Scope Notes and Security Boundaries

Scope Note ID	Change Type	Scope Boundary
NIS-S N01	Modified	The model migration changes the provider, model, runtime, and integration wording only. Existing approved AI-supported business functions remain unchanged within their approved boundaries. Any earlier sponsor or Project Plan wording suggesting comprehensive, broad, or unrestricted data collection is superseded where inconsistent by the latest approved data-minimisation, privacy, visibility, purpose-limitation, and Backend-control requirements. Only the minimum permitted data required for the current approved request or administrative operation may be processed.
NIS-S N02	Added	No automatic model switching, automatic model failover, runtime routing, load-based selection, per-feature assignment, or user model selection is introduced. Qwen3.5-4B is the manually configured development and low-spec alternative model.
NIS-S N03	Added	vLLM, SGLang, and llama.cpp are future deployment options. GPU selection, server specification, containers, orchestration, distributed inference, and production topology remain Deferred.
NIS-S N04	Added	This package defines design-level requirements, responsibility boundaries, and preparation specifications. It does not claim completed software implementation and does not define API endpoints, routes, methods, request or response schemas, database collections, fields, model installation, deployment scripts, or implementation code.
NIS-S N05	Added	Exact Interest Tag validation rules, including required fields, format, character limits, whitespace handling, duplicate detection, normalisation, search behaviour, and storage structure, are not defined in this Sprint.

NIS-S N06	Added	The Admin scope is limited to adding new Interest Tags to the existing Interest catalogue. "Interest Tag" refers to the existing general Interest catalogue and does not define an Event-specific or Activity-specific entity. Standalone Interest Read administration, Update, Rename, Merge, Deactivate, Reactivate, Delete, Archive, Restore, Interest content review, approval, content-rejection decisions, moderation, User Account Administration, account moderation, and access to private user data are outside this Sprint. Backend rejection of an unauthorised, invalid, or out-of-scope Create request is an operational security and validation outcome, not an Interest content-review rejection decision.
--------------	-------	--

NIS-S N07	Added	No Admin UI, dashboard, or mobile page is required. Any earlier Project Plan reference to an Admin prototype, Admin configuration interface, dashboard, mobile page, or complete Admin UI workflow is Deferred and is not part of the current New Incremental Sprint. The restricted internal script or command-line tool and protected internal Admin API support Interest Tag creation within the existing Interest catalogue only.
--------------	-------	---

NIS-S N08	Modified	Existing Check-in, Privacy, Event, Friends Activity, Special Care, Friend Search, Recommendation, Homepage, Persona, Going, and Interested business rules remain frozen. The only approved catalogue-level synchronisation is that a successfully added Interest Tag becomes available to existing features that already retrieve, display, select, or use the Interest catalogue under their existing approved rules. This includes existing Check-in Interest Tag use and existing Interest-based Event and Friend Recommendation functions where applicable. No existing records, selections, Event associations, Check-in records, Profiles, recommendation candidates, weights, scores, rankings, AI inputs, or historical data are automatically changed. Where earlier Project Plan wording conflicts with the latest approved Sprint 5 Privacy flow, the latest Sprint 5 decision takes precedence: initial Accept saves Public, initial Reject saves Private, either choice permits application access, Friends-only remains available later through Settings, Check-in automatically applies the saved privacy state, and location has no separate visibility default or independent sharing toggle.
--------------	----------	--

7. Cross-Team Handoff

7.1 Member B / Business Analyst

Member B / Business Analyst shall define, document, and maintain:

- the New Incremental Sprint design-level requirements and temporary NIS identifiers;
- consistent removal of Gemini-specific terminology;
- Qwen model, runtime, Adapter, manually configured Qwen3.5-4B terminology, and deterministic Backend fallback terminology;
- the deterministic Recommendation and Qwen responsibility boundary;
- the Interest Tag Addition requirement and its use of the existing Interest catalogue;
- the boundary that no Event-specific or Activity-specific tag entity is introduced;
- the Create-only Admin scope;
- the synchronisation boundaries for existing Interest-consuming features;
- the rule that newly created Interest Tags are not automatically assigned to users, Events, Check-ins, recommendations, AI input, Profiles, or historical records;
- the exclusion of Interest content review, approval, content-rejection decisions, moderation, broader Interest administration, and User Account Administration, while preserving the Backend's operational rejection of unauthorised, invalid, or out-of-scope Create requests;

- the Admin authentication, authorisation, entity-specific validation, creation-audit, protected Backend access, and no-UI boundaries;
- the latest Sprint 5 Privacy precedence over inconsistent earlier Project Plan wording;
- the latest data-minimisation precedence over inconsistent earlier sponsor or Project Plan wording;
- consistency with frozen Sprint requirements.

7.2 Flutter / Client

Flutter / Client shall produce and maintain the Client-side design specification that:

- sends approved AI requests only to the Backend;
- displays only Backend-approved AI or deterministic Backend fallback results;
- does not directly access Qwen, Ollama, vLLM, SGLang, llama.cpp, or model endpoints;
- does not implement automatic model switching or automatic model failover;
- does not store model credentials or protected Backend secrets;
- uses the updated existing Interest catalogue in screens that already display or select Interest Tags;
- does not introduce an Event-specific or Activity-specific tag model;
- does not automatically select or assign newly created Interest Tags;
- does not redesign existing Check-in, Profile, Event Recommendation, Friend Recommendation, or other Interest-consuming workflows;
- does not include an Admin UI, Admin dashboard, Admin mobile page, Interest-review interface, moderation workflow, or User Account Administration interface in the current Sprint;
- treats any earlier Project Plan Admin prototype or configuration-interface concept as Deferred;
- does not declare or determine Admin authority on the Client;
- does not expose the restricted internal Admin API as an ordinary user function.

7.3 AI / Algorithm

AI / Algorithm shall define and document:

- approved prompt and structured-output behaviour for Qwen3.5-9B;
- Qwen3.5-4B only as the manually configured development and low-spec alternative model, not as automatic model failover;
- compatibility with Backend validation and data-minimisation requirements;
- the rule that all recommendation candidates, weights, scores, and final ordering are generated by the deterministic Backend recommendation algorithm;
- the restriction that Qwen may only explain, summarise, or conversationally present Backend-approved recommendation results;
- the rule that newly added Interest Tags may be used only when supplied through existing Backend-approved Interest and Recommendation flows;
- the prohibition on Qwen independently creating, selecting, assigning, weighting, scoring, or ranking Interest Tags;

- the prohibition on Qwen adding candidates, modifying weights, overriding scores, changing final ordering, accessing the database, or exercising state-changing authority;
- replacement of Gemini-specific prompt and context wording with Qwen-compatible wording.

7.4 Backend

Backend shall define and document:

- the Backend design for Qwen Local Model Adapter and the current Ollama integration;
- the layered service, data-access, validation, data-minimisation, purpose-limitation, and deterministic Backend fallback responsibilities;
- removal of Gemini API, credential, quota, billing, SDK, and Provider Adapter dependencies;
- authentication, authorisation, privacy, visibility, request construction, output parsing, and output-validation requirements;
- the rule that inconsistent earlier broad or unrestricted sponsor or Project Plan data-collection wording is superseded by current minimum-permitted-data requirements;
- the deterministic Recommendation boundary under which the Backend generates all recommendation candidates, weights, scores, and final ordering;
- the rule that Qwen receives only Backend-approved results or context and cannot alter recommendation candidates, weights, scores, or ordering;
- manual Admin assignment through Firebase Authentication and Backend-verified Firebase custom claims;
- the design and scope of a protected Create operation for adding an Interest Tag to the existing Interest catalogue;
- rejection of any request to create a separate Event-specific or Activity-specific Interest entity or catalogue;
- restricted internal script or command-line access to the protected Create operation;
- enforcement of the Create-only Admin scope;
- entity-specific validation before Interest Tag creation;
- creation of the new Interest Tag within the existing Interest catalogue;
- availability of the updated catalogue through existing catalogue retrieval paths;
- preservation of existing Check-in, Profile, Event Recommendation, Friend Recommendation, privacy, visibility, and other Interest-consuming business logic;
- prevention of automatic assignment to users, Events, Check-ins, recommendations, AI input, Profiles, or historical records;
- prevention of retrospective data changes or migration caused solely by Interest Tag creation;
- separate audit-field applicability for successful Interest Tag creation and failed or operationally rejected Create requests, ensuring that only successful creation references a created target record;
- recording of attempted Create operation, available target information, attempted tag value where permitted, timestamp, and failed or operationally rejected outcome for unsuccessful requests without claiming successful tag creation;

- operational rejection and audit handling for unauthorised, invalid, or out-of-scope Interest Tag Create requests;
- exclusion of formal Interest Read administration, Update, Rename, Merge, Deactivate, Reactivate, Delete, Archive, Restore, Interest content review, approval, content-rejection decisions, moderation, and User Account Administration from the Admin design scope;
- prevention of Admin access to private user data;
- prevention of direct Admin and AI access to Firestore;
- treatment of vLLM, SGLang, and llama.cpp as future deployment options only.